AI-Powered SRE Agents for Azure Container Apps (2025–Present)

Problem: Azure Container Apps incidents required manual root cause analysis, leading to long mitigation and resolution times for customer-facing issues; agent accuracy, deployment reliability, and auto-remediation coverage needed sustained investment.

What I Did:

- Designed and deployed AI-powered diagnostic agents for ACA incident resolution — ICM Summarization Agent, CoreDNS Agent, Sessions Agent, and Billing Subagent.
- Shifted architecture from workflow-based to investigative prompts, dramatically improving AI-aligned RCA accuracy; closed 20+ agent backlog items through targeted ICM analysis.
- Drove a series of experiments lifting RCA accuracy and visibility — agent now posts an RCA on ~99% of incidents; on-call rates root cause correct on ~76% over a 30-day window.
- Auto-resolved 57 of ~400 CRIs and authored ~132 repair items in a six-month window through evolving auto-repair capabilities (compare agent-generated RCA vs actual RCA, create repair items, auto-resolve qualifying incidents).
- Extended subagent skills with concrete customer-incident patterns: SAL-deletion, Geneva 503, env-delete with orphaned load balancers.
- Stood up the Ev2 deployment pipeline for the agent itself, shipping through standard Azure release machinery instead of manual updates that previously caused conflicts when changes landed close together.
- Presented the RCA Agent at the IDC Pinnacle event; squad won the IDC Pinnacle FY'26 **best AI success story award** among ~200+ teams.

Impact: Reduced ACA incident P50 time-to-mitigate by 97% (185 → 6 min) and P50 time-to-resolve by 64% (269 → 98 min). Sustained ~99% RCA coverage with ~76% on-call-rated accuracy; auto-resolved 57 CRIs and produced 132 repair items in six months. Recognized as a hero example across the org.

Foundry Control Plane Integration with ACA (2026)

Problem: Agentic apps hosted on ACA were not discoverable or manageable from the Foundry Control Plane, making ACA a second-class hosting target for agents in Foundry.

What I Did:

- Designed and shipped the integration that lets Foundry Control Plane (FCP) discover and surface agentic apps on ACA — wired through a **new preview API across clouds, CP, and CRD**.
- Supported both **customer-declared** and **auto-detection** modes so customers retain control while coverage still extends to apps that don't self-declare.
- Covered ~1,200–2,000+ potential agentic apps on ACA, making ACA a first-class hosting target for agents in Foundry.
- Although the upstream Foundry integration was later shelved due to Foundry re-architecture, the ACA-side foundation laid significant observability into agentic workload patterns on ACA.

Impact: Established the integration surface and agent-workload observability that positions ACA as the canonical hosting target for agentic apps; design decisions (declared + auto modes) preserved customer control while maximizing coverage.

Express Apps on Azure Dev Compute (ADC) — Public Preview Enablement (2026)

Problem: ACA Express Apps (sub-500ms ephemeral sandbox offering on the new ADC platform layer) was blocked from Public Preview by missing parity with standard ACA — specifically ephemeral volume support and Log Analytics customer-log delivery for the 100K–250K ACA environments using LA today.

What I Did:

- Closed two of the largest gaps holding back Express Public Preview — **ephemeral volume support** and **Log Analytics customer-log delivery** — by landing changes on both the ADC and ACA sides so the end-to-end flow works through a single customer-facing surface.

• Drove a detailed design discussion with the ADC team on the correct way to add LA Collector API support — evaluating ADC-native vs an ACA-specific sidecar model — and got sign-off from ADC and other key stakeholders **before any code landed**, avoiding future churn.

• Reused ADC's existing volume infrastructure for ACA ephemeral-storage parity instead of adding a new kind, keeping the integration small and shipping in-window.

• Adapted design across multiple iterations as the underlying ADC architecture changed, maintaining best customer experience.

Impact: LA delivery is the migration path for the ~100K–250K ACA environments using LA today, and the P0 migration blocker for Express Public Preview (June 2026). Express migration targets **~$40M annual COGs savings** through core reclaim.

Azure Container Apps Sessions (2024–2025)

Problem: ACA Sessions lacked observability, had critical async execution bugs, and needed data-plane APIs for internal and external customers.

What I Did:

• Built Session Management APIs (Delete, List, Get) to improve session observability and control.

• Fixed critical async execution bugs in code interpreter sessions and Node.js session-specific issues.

• Enabled early session deletion — a key requirement from the Fabric Team — reducing billing and costs.

• Added performance tests for ACA Sessions, identifying code execution regressions.

• Enabled .NET code interpreter support for the Security Copilot Team.

• Implemented Geneva actions to manage upgrades for Node.js sessions.

Impact: Improved session stability and experience for ~300 external customers actively using ACA Sessions. Enabled cost optimization for ACA session customers through early deletion support.

Azure Functions on Azure Container Apps — Reliability & Platform Capability (2025–Present)

Problem: Azure Functions on ACA lacked feature parity with native Azure Functions and had reliability gaps around graceful shutdown, trigger fidelity, and authentication — surfacing as multiple CRIs with the same error across ~5K active apps.

What I Did:

• Designed and shipped **graceful drain** for managed function apps so in-flight executions complete cleanly during scale-in — retired three separate CRIs that all stemmed from the same underlying error.

• Delivered **queueLength override** for queue triggers and a broader **scale-rule override** path, lifting trigger fidelity for managed Function and Logic apps.

• Added **JWT clock-skew leeway** to harden auth stability for managed Function and Logic apps.

• Implemented function keys management commands for ACA Functions.

• Added invocation tracing/summary and durable-events logging for diagnostics.

• Added support for editing KEDA scaling rules in Azure Functions on ACA.

• Authored API spec for Function ARM APIs.

• Closing feature-parity gaps between Azure Functions on ACA and native Azure Functions on an ongoing basis.

Impact: Stabilized ~5K active Functions-on-ACA apps and ~1,322 external Functions apps to the point where the most frequent CRIs are gone; lifted trigger fidelity and auth stability across managed Function and Logic apps.

Functions Observability Migration to Central Kusto (2026)

Problem: Functions telemetry was emitted to the legacy waws Kusto clusters owned by App Service, creating storage pressure on the old clusters and fragmenting the query surface used by alerting, customer detectors, and agent skills.

What I Did:

- Routed **FunctionsLogs**, **FunctionsMetrics**, and **DurableFunctionsEvents** from the legacy waws Kusto clusters into the modern **capps** clusters.
- Persisted through a non-trivial issue on the **Legion** side for consumption function apps where logs were not flowing, going deeper into the problem until it was resolved.
- Unblocked alerting, customer detectors, and many agent skills through a single query surface.

Impact: Resolved storage pressure on the legacy App Service Kusto clusters and consolidated the query surface for alerting, customer detectors, and SRE-agent skills.

Open-Source Contributions to Dapr — CNCF Project (2021–2024)

Problem: Dapr, a CNCF open-source project for distributed applications, needed components upgraded to stable, new features, improved developer experience, and active community support.

What I Did:

- Upgraded MQTT PubSub component from alpha to stable; contributed fix to upstream library; added certification tests.
- Marked Azure CosmosDB state store, Configuration API building block, and Redis/PostgreSQL configuration stores to stable.
- Implemented cascade terminate and purge for Dapr Workflows — changes spanning runtime and multiple SDKs.
- Added bulk subscribe support in Azure Event Hubs and Dapr Workflow performance tests; identified several P0 bugs.
- Enabled airgap installation support; built MSI installer; made Dapr available in winget with automated package updates.
- Published combined coverage reports for conformance and certification tests.
- Resolved customer issues across GitHub (Runtime, Components-Contrib, SDKs) and Discord.

Impact: Accelerated Dapr component maturity and improved developer experience, driving broader adoption of Dapr across Azure and the open-source community.

Dapr Internal Fork & CI/CD Infrastructure (2021–2024)

Problem: Microsoft's internal services (AKS, Container Apps) needed a reliable internal fork of Dapr with fast release turnaround, compliance, and multi-architecture support.

What I Did:

- Architected and maintained Dapr internal fork across 5 repositories (dapr, cli, dashboard, components-contrib, kit) with build, code-signing, and release pipelines.
- Automated OSS-to-internal merge pipeline, achieving internal releases within <1 day SLA of OSS release.
- Re-designed internal repository to a patch-based model with separate per-service images for faster, targeted releases.
- Migrated CI/CD pipelines to 1ES (OneBranch) for compliance; integrated Ev2 deployment for Standard Deployment Practices.
- Added multi-arch support (ARM64, ARM, AMD64), Mariner and distroless images; handled 1P Container Registry migration.
- Delivered nightly builds for Azure Container Apps.

Impact: Enabled AKS and Container Apps to reliably consume internal Dapr images with rapid turnaround, supporting seamless adoption of Dapr across Azure distributed services.

Dapr AKS Extension — Security & Multi-Region Rollouts (2022–2026)

Problem: Dapr AKS Extension customers needed secure, up-to-date versions across all Azure regions with fast CVE remediation, reliable rollouts, and fresh artifacts.

What I Did:

- Led Dapr 1.15 and 1.16 rollouts across all Azure regions; shipped additional security-vulnerability fix releases.
- Drove subsequent extension version through prod and added a **daily extension-only rollout** so artifacts stay fresh.
- Fixed multiple CVE/security vulnerabilities; remediated issues across ~400 AKS clusters.
- Expanded AKS extension availability to new regions (e.g., AustriaEast) based on customer demand.
- Integrated CodeQL into AKS Extension pipeline for security and code quality.
- Consolidated multiple pipelines into a single OneBranch-compliant pipeline to expedite releases.
- Resolved customer-reported AKS extension ICMs promptly.

Impact: Upgraded ~260 subscriptions and ~400 AKS clusters to secure Dapr versions; sustained fresh-artifact delivery via daily rollouts. Workstream subsequently offloaded to allow focus on higher-impact areas.

On-Call Reliability & DRI Operations (2024–Present)

Problem: Azure Container Apps required consistent, high-quality on-call coverage to maintain service uptime and resolve customer-impacting incidents rapidly across CRIs and LSIs.

What I Did:

- Served as on-call DRI across multiple rotations — resolved **38 CRIs and 84 LSIs (36 Sev2s)** in the most recent 6-month window alone, on top of **200+ prior incidents including 27+ CRIs**.
- Handled 36 Sev2 LSIs (P50 TTM 1h, P50 TTR 25h vs ACA baseline 3.9h/60h) and 6 Sev3 CRIs in earlier windows, consistently outperforming team baselines.
- Delivered repair items to address root causes, reducing incident recurrence and improving system resilience.
- Improved alert fidelity by updating 5 LSI alerts based on investigation findings.
- Fed agent feedback and skill updates back from incidents the agent struggled on so its accuracy keeps improving — actively listening in weekly IcM discussions even off-shift.

Impact: Ensured high service uptime and reduced operational noise through proactive issue resolution, consistently exceeding ACA on-call baselines while closing the agent→incident feedback loop.

Azure Key Vault Integration & Security Posture (2025–2026)

Problem: ACA Managed Environment Storage API was handling raw connection strings in ~27K daily API calls, creating a security risk; the Dapr repo also needed proactive vulnerability scanning.

What I Did:

- Integrated Azure Key Vault with Managed Environment Storage API, enabling secure secret handling.
- Eliminated raw connection strings in storage create API calls.
- Resolved CodeQL violations across container apps, improving code security and compliance.
- Used **Argus security-scan tool** to scan the Dapr repo for vulnerabilities; shared the report with the team along with action items to fix the highest-risk ones.

Impact: Provided a secure alternative to raw account keys for ~27K daily storage API calls, strengthening security posture and aligning with enterprise-grade standards; proactively surfaced Dapr-repo vulnerabilities for team remediation.

AI-Augmented Engineering & Craft (2026)

Problem: As Copilot accelerated authoring, the engineering bottleneck shifted to **verification** — behavioural diffs in a multi-tenant platform can have wide blast radius, and a drain feature regression (revision regression + nil-dereference Sev3) reinforced the need for disciplined self-review and end-to-end validation before merge.

What I Did:

- Incorporated **Copilot self-review** on PRs with three parallel perspectives — *Advocate* defending intent, *Skeptic* hunting regressions and edge cases, *Architect* evaluating direction — to catch issues before peer review. Used the same skill on peer PRs as a deeper second-opinion pass.
- Used Copilot across the full flow — design → implementation → PR review — and made **e2e validation on a private stamp** a standard step before merging any meaningful feature change.
- Adopted a "what does this change for every existing customer" lens on every PR after the drain-feature regression, reasoning blast radius up-front instead of relying on post-merge detection.
- Delivered **62 PRs** across Functions, Sessions, and AKS-extension components in a single review cycle, sustained by AI-augmented authoring and review.
- Kept the codebase tidy: removed dead types, aligned client contracts with service changes, and added docs in the same PR as design changes.
- Shared learnings on AI tooling with the broader AppPlat team to lift team-wide AI fluency.

Impact: Shifted personal engineering practice to match the AI-augmented era — author fast, verify deeply. Sustained high PR throughput (62 PRs / cycle) while improving the quality of self-review and reducing post-merge surprises.

Mentoring & Cross-Team Leadership (2025–2026)

Problem: Sustaining team velocity at SDE2 level requires unblocking newer engineers, building cross-team relationships that make follow-on features easier to ship, and absorbing on-call asymmetry when teammates need coverage.

What I Did:

- Mentored new hires **Khushi and Divyansh** through their first on-call rotations — helping with faster customer resolution and unblocking issues during both on-call and dev flows.
- Continued mentoring Khushi separately on agent-platform and scaling-rules PRs, and gave a junior engineer space to learn and own parts of the Functions-on-ACA feature set.
- Built durable cross-team relationships with **ADC, Functions, FCP, and SRE-Agent teams** to land cross-org features end-to-end — those review relationships make the next feature easier to ship.
- Collaborated with the Playwright testing team on a proof of concept using ACA Sessions.
- Took ad-hoc on-call shifts to support the China team during new-year leave and to cover teammates whose plans changed; stayed engaged in CRI/LSI discussions off-shift.
- Presented the ACA RCA Agent at the IDC Pinnacle event; squad won the FY'26 best AI success story award among ~200+ teams.

Impact: Multiplied team impact by unblocking first-time on-call engineers, building review relationships across ADC/Functions/FCP/SRE-Agent that compound over time, and absorbing on-call coverage gaps proactively.